



Flora (꽃담) 프로젝트

원리 · 흐름 정리노트

팀원이 함께 읽으며 "왜 이렇게 돌아가는가"를 이해하기 위한 학습 자료

Spring Boot 3.5 · React 19 · Oracle XE · MongoDB

Team 積み木

이 노트를 읽는 법

이 문서는 "Flora가 어떻게 돌아가는가"를 팀원 모두가 이해하는 것을 목표로 합니다. 단순히 기능이 뭐가 있는지가 아니라, 요청이 들어오면 내부에서 무슨 일이 벌어지는지 — 그 원리를 따라갑니다. 한번에 다 외울 필요는 없습니다. 개념 → 원리 → 우리 프로젝트 예시 순서로 읽으면서, 나중에 필요할 때 해당 장을 다시 찾아오면 됩니다.

권장 학습 순서

1. 1장·2장: 웹 서비스가 왜 이런 구조로 만들어지는지 큰 그림을 먼저 잡습니다.
2. 3장·4장: HTTP와 REST — 프론트와 백엔드가 대화하는 규칙을 익힙니다.
3. 5장·6장: 백엔드 내부 구조(Controller/Service/Repository)와 JPA가 실제로 무엇을 하는지.
4. 7장: JWT 로그인은 어떻게 "로그인 상태"를 유지하는가.
5. 8장: React가 왜 SPA인지, 페이지 이동은 어떻게 되는가.
6. 9장: Oracle과 MongoDB를 함께 쓸 때의 원리.
7. 10장: 실제 요청 하나가 DB까지 닿는 과정을 E2E로 따라갑니다 (가장 중요).
8. 11장: 외부 API 호출 패턴.
9. 12장: 자주 나오는 패턴과 용어 정리, 공부 로드맵.

Tip. 이 노트를 읽을 때는 우리 프로젝트 코드를 옆에 켜두세요. 설명 뒤에 "Flora에서는 ____.java 파일을 보세요" 식으로 짚어드립니다. 실제 파일을 열어 본문과 대조하면 원리가 훨씬 빠르게 체화됩니다.

1장. 큰 그림 — 웹 서비스는 왜 세 조각으로 나뉘는가

1-1. 세 조각 (3-Tier)

모든 현대 웹 서비스는 세 개의 역할로 나뉩니다. Flora도 마찬가지입니다.

- 프론트엔드 (Frontend) — 사용자가 보는 화면. 브라우저에서 실행되는 React 앱.
- 백엔드 (Backend) — 사용자의 요청을 처리하는 서버. Spring Boot 애플리케이션.
- 데이터베이스 (Database) — 데이터를 영구 보관하는 저장소. Oracle + MongoDB.

1-2. 왜 굳이 셋으로 나누는가

하나의 프로그램에 다 넣어도 되지 않을까? 하지만 분리하는 이유가 명확합니다.

- 역할 분리: 화면 로직은 프론트, 비즈니스 로직은 백엔드. 코드가 섞이면 수정이 어려워집니다.
- 보안: DB를 브라우저에서 직접 만지면 사용자가 아무 데이터나 조작할 수 있습니다. 백엔드가 "문지기" 역할을 합니다.
- 확장성: 사용자가 많아지면 백엔드만 서버를 여러 대 둘 수 있습니다. 프론트는 CDN으로, DB는 복제본으로 따로 키웁니다.
- 협업: 프론트 개발자와 백엔드 개발자가 동시에 다른 파일을 수정할 수 있습니다.

1-3. Flora의 실제 구성

계층	우리가 쓰는 기술	실제 위치
프론트엔드	React 19 + Vite + axios	frontend/kkotdam/
백엔드	Spring Boot 3.5 + Spring Security	backend/src/main/java/com/flora/backend/
RDB	Oracle XE (JPA)	application.properties의 datasource 설정
NoSQL	MongoDB (Spring Data)	같은 설정 파일의 mongodb 설정

SPA — Single Page Application. 페이지 전환 때마다 서버에서 HTML을 새로 받지 않고, JavaScript가 화면을 갈아 끼우는 방식. Flora의 프론트는 React Router로 SPA를 구현합니다.

REST API — 백엔드가 제공하는 "데이터 창구". 프론트는 `/api/products` 같은 주소로 요청을 보내고 JSON 데이터를 받습니다. 자세한 건 3장에서.

2장. 한 번의 클릭이 만드는 10단계 여행

사용자가 "장바구니에 담기" 버튼을 눌렀을 때 내부에서 벌어지는 일을 10단계로 정리했습니다. 이 흐름은 Flora 어디에서 버튼을 누르든 기본적으로 동일합니다.

10. 사용자가 상품 상세 페이지에서 "장바구니" 버튼을 클릭합니다.
11. React 컴포넌트의 onClick 핸들러가 실행됩니다 (예: ProductDetailPage.jsx).
12. 핸들러는 cartApi.addItem(productId, qty) 함수를 호출합니다 (src/api/ 아래).
13. cartApi는 내부에서 axios.post('/api/cart', {...}) 요청을 보냅니다.
14. axios 인터셉터가 저장된 JWT 토큰을 Authorization 헤더에 자동으로 붙입니다.
15. 요청이 백엔드의 Spring Security 필터에 도달합니다. 필터는 토큰을 검증해 "누가 보낸 요청인지"를 파악합니다.
16. CartController의 @PostMapping 메서드가 요청을 받습니다.
17. Controller는 CartService.addItem(...)을 호출해 비즈니스 로직을 위임합니다.
18. Service는 CartItemRepository를 통해 Oracle DB에 INSERT 또는 UPDATE를 수행합니다.
19. 결과가 DTO 형태로 JSON 변환되어 응답 → axios가 받아 → 프론트가 화면을 갱신합니다.

Tip. 이 10단계를 외워두면, 이후 모든 챕터가 "그 10단계 중 어느 부분에 대한 상세 설명"인지 머릿속에 자리 잡습니다.

3장. HTTP와 REST API — 프론트·백엔드 대화 규칙

3-1. HTTP가 뭐길래

HTTP는 웹에서 클라이언트(브라우저)와 서버가 대화하는 약속입니다. 모든 대화는 "요청(Request)"과 "응답(Response)"으로 이루어집니다.

요청은 4가지로 구성

- 메서드(Method): GET(조회), POST(생성), PUT(수정), DELETE(삭제)
- 경로(Path): /api/products, /api/orders/123
- 헤더(Headers): Content-Type, Authorization 등 부가정보
- 바디(Body): JSON 데이터 (POST, PUT일 때)

응답도 마찬가지로

- 상태 코드(Status): 200(성공), 201(생성됨), 400(잘못된 요청), 401(인증 안 됨), 404(없음), 500(서버 오류)
- 헤더, 바디(JSON 데이터)

3-2. REST란

REST는 "URL과 HTTP 메서드를 규칙적으로 써서 API를 설계하자"는 원칙입니다. 핵심 아이디어는 단순합니다.

- 리소스(자원)는 URL로 표현한다. 상품은 /products, 주문은 /orders.
- 무엇을 할지는 HTTP 메서드로 표현한다. GET /products(목록), POST /products(등록), DELETE /products/5(5번 삭제).

3-3. Flora의 REST 설계

기능	메서드	경로	처리 컨트롤러
회원가입	POST	/api/auth/signup	AuthController
로그인	POST	/api/auth/login	AuthController
상품 목록	GET	/api/products	ProductController
상품 상세	GET	/api/products/{id}	ProductController
장바구니 담기	POST	/api/cart	CartController

기능	메서드	경로	처리 컨트롤러
주문 생성	POST	/api/orders	OrderController
게시글 작성	POST	/api/community/posts	CommunityController
식물 진단	POST	/api/plants/diagnosis	(관련 컨트롤러)

Tip. 우리 프로젝트는 Swagger를 달아두었습니다. 백엔드를 실행하고 <http://localhost:8080/swagger-ui.html> 에 접속하면 모든 API 목록과 테스트 UI가 자동 생성됩니다.

3-4. JSON 한 번 더 짚기

JSON은 그저 데이터를 글자로 주고받기 위한 포맷일 뿐입니다. { "key": "value" } 이런 식. 자바 객체나 자바스크립트 객체를 이 문자열로 바꿔 보내고, 받는 쪽에서 다시 객체로 푸는 것이죠. Spring Boot는 Jackson 라이브러리가, 브라우저는 axios가 자동으로 해줍니다.

4장. Spring Boot 3계층 — Controller · Service · Repository

4-1. 왜 계층을 나누는가

요청 처리 코드를 한 파일에 몰아두면, "장바구니 담기" 하나를 수정하려 해도 수백 줄의 파일을 뒤져야 합니다. Spring Boot 진영에서는 관례적으로 다음 3계층으로 나눕니다.

계층	책임	하면 안 되는 일
Controller	HTTP 요청 수신·응답. 어떤 URL로 들어왔는지만 신경쓴다.	DB 조회, 비즈니스 로직 직접 구현
Service	비즈니스 로직. 주문 생성 규칙, 쿠폰 계산, 포인트 적립 등.	요청·응답 포맷을 의식하는 코드
Repository	DB 접근. SQL 또는 JPA 메서드로 저장·조회만.	로직 판단, 사용자 권한 검사

4-2. Flora의 인증 기능으로 보는 3계층

회원가입 흐름을 세 계층으로 따라가 봅시다.

Controller — AuthController.java

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/signup")
    public ResponseEntity<UserDto> signup(@Valid @RequestBody UserDto req) {
        return ResponseEntity.ok(authService.signup(req));
    }
}
```

포인트: Controller는 사실상 "입구"일 뿐입니다. 요청 본문을 받아 Service에 넘기고, 결과를 ResponseEntity로 감싸 돌려주는 게 전부입니다.

Service — AuthService.java

(실제 코드는 더 길지만 핵심만)

```
public UserDto signup(UserDto req) {
    if (userRepository.existsByEmail(req.getEmail())) {
        throw new IllegalStateException("이미 가입된 이메일");
    }
    User user = User.builder()
```



```

        .email(req.getEmail())
        .passwordHash(passwordEncoder.encode(req.getPassword()))
        .nickname(req.getNickname())
        .role("USER")
        .build();
        User saved = userRepository.save(user);
        return UserDto.from(saved);
    }

```

포인트: "이미 가입된 이메일인지 체크", "비밀번호는 해시로", "기본 권한은 USER" 같은 비즈니스 규칙이 여기 모입니다.

Repository — UserRepository.java

```

public interface UserRepository extends JpaRepository<User, Long> {
    boolean existsByEmail(String email);
    Optional<User> findByEmail(String email);
}

```

포인트: 인터페이스만 있습니다. JpaRepository를 상속받으면 save(), findById() 같은 기본 메서드가 공짜로 생깁니다. existsByEmail 같은 이름 규칙을 따르면 JPA가 알아서 SQL을 만들어줍니다.

4-3. 의존성 주입(DI)과 @RequiredArgsConstructor

AuthController가 AuthService를 "가져다 쓰는" 방법이 바로 DI(Dependency Injection)입니다. 우리 코드는 Lombok의 @RequiredArgsConstructor를 붙여 final 필드에 대한 생성자를 자동 생성하고, Spring이 그 생성자에 맞는 Bean을 넣어줍니다. 이렇게 하면 테스트할 때 가짜 Service를 주입하기도 쉽습니다.

△ **주의.** Service가 Repository를 "new"해서 만들지 마세요. @Service · @Repository로 선언하면 Spring이 싱글톤으로 관리해줍니다. new로 직접 만들면 트랜잭션 · 프록시 기능이 꺼집니다.

5장. JPA — 자바 객체가 어떻게 DB 테이블이 되는가

5-1. ORM이 뭐야

ORM(Object-Relational Mapping)은 자바 객체와 관계형 DB 테이블 사이의 번역기입니다. 개발자는 User 객체를 다루기만 하면, 뒤에서 JPA가 INSERT/UPDATE/SELECT SQL을 만들어 실행합니다. JPA는 그 인터페이스 규약이고, 실제 구현체로 Hibernate가 쓰입니다.

5-2. Entity = 테이블

User.java는 USERS 테이블과 매핑됩니다.

```
@Entity
@Table(name = "USERS")
@Getter @Setter @Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // PK
    @Column(unique = true)
    private String email;
    private String passwordHash;
    private String nickname;
    private String role; // USER / SELLER / ADMIN
    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

포인트: @Entity 한 줄로 USERS 테이블에 자동 매핑됩니다. @Id는 PK, @Column(unique=true)는 UNIQUE 제약, @CreationTimestamp는 INSERT 시 자동으로 현재 시간 채우기.

5-3. 관계 매핑

Product는 seller(User)를 갖고, Order는 product와 user를 참조합니다.

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "seller_id")
private User seller;
```

- @ManyToOne: 여러 상품이 한 판매자에 속한다.
- fetch = LAZY: 상품을 조회했다고 판매자까지 자동으로 꺼내오지 않고, .getSeller()를 호출할 때만 DB에 추가 쿼리로 가져온다.

△ **주의.** N+1 문제 — 리스트를 돌며 각 항목의 연관 엔티티를 계속 조회하면 쿼리가 N+1번 나갑니다.
해결: JPQL의 JOIN FETCH 또는 @EntityGraph 사용.

5-4. Repository 쿼리 메서드 규칙

JPA는 메서드 이름을 해석해 SQL을 자동 생성합니다.

메서드 이름	자동 생성되는 SQL (개념)
findByEmail(String email)	SELECT * FROM users WHERE email = ?
findByRoleAndIsActive(String r, boolean a)	SELECT * FROM users WHERE role = ? AND is_active = ?
existsByEmail(String email)	SELECT COUNT(*) > 0 FROM users WHERE email = ?
findTop10ByOrderByCreatedAtDesc()	SELECT * FROM users ORDER BY created_at DESC FETCH FIRST 10 ROWS

Tip. 복잡한 쿼리는 @Query 어노테이션으로 JPQL이나 Native SQL을 직접 쓸 수 있습니다.

6장. DTO — 엔티티를 그대로 노출하지 않는 이유

Entity를 API 응답으로 그대로 내보내면 세 가지 문제가 생깁니다.

- 20. 보안: passwordHash 같은 민감 필드가 응답에 섞여 나갈 수 있음.
- 21. 결합도: DB 스키마를 바꾸면 API 스펙까지 자동으로 바뀌어 프론트가 깨짐.
- 22. 직렬화 순환: 양방향 관계가 있으면 JSON이 무한 중첩됩니다.

그래서 우리는 각 엔티티마다 UserDto, ProductDto 식의 DTO를 두고, DTO.from(Entity) 정적 메서드로 변환합니다.

```
public class UserDto {
    private Long id;
    private String email;
    private String nickname;
    private String role;
    // passwordHash는 절대 포함하지 않음

    public static UserDto from(User u) {
        return UserDto.builder()
            .id(u.getId())
            .email(u.getEmail())
            .nickname(u.getNickname())
            .role(u.getRole())
            .build();
    }
}
```

Tip. 요청용·응답용 DTO를 분리해도 됩니다. 예: SignupRequestDto / UserResponseDto. Flora는 단순화를 위해 단일 DTO로 시작했고, 나중에 분리할 수 있게 구조만 열어뒀습니다.

7장. JWT — "로그인 상태"는 어떻게 유지될까

7-1. 문제 상황

HTTP는 본래 Stateless(상태를 기억하지 않음)입니다. 서버 입장에서 매 요청은 처음 보는 사람이 보낸 것처럼 보입니다. 그럼 "이 사람이 로그인한 회원 홍길동"이라는 사실은 어떻게 유지할까요?

7-2. 세션 방식 vs JWT 방식

방식	원리	특징
세션	서버가 sessionId를 발급, 본인 메모리에 저장	서버 재시작·스케일아웃 시 관리 부담
JWT	토큰 자체에 사용자 정보가 서명되어 들어감	서버가 상태를 저장할 필요 없음 (stateless)

Flora는 JWT를 택했습니다.

7-3. JWT의 구조

JWT는 세 조각이 점(.)으로 구분된 문자열입니다.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 (Header)
.eyJzdWIiOiJyMywicz9sZSI6IlVTRVIifQ (Payload)
.AbCdEf...signature... (Signature)
```

- Header: 서명 알고리즘(HS256 등)
- Payload: userId, role, 만료시각(exp) 같은 정보. Base64 인코딩이므로 누구나 내용을 읽을 수 있음.
- Signature: Header+Payload를 서버의 비밀키로 서명. 변조 방지용.

△ **주의.** JWT는 "암호화"가 아니라 "서명"입니다. 민감 정보(비밀번호 등)를 Payload에 담지 마세요.

7-4. 전체 흐름

23. ① 프론트가 POST /api/auth/login에 이메일·비밀번호 전송.
24. ② AuthService가 DB에서 사용자를 찾아 passwordHash와 비교 검증.
25. ③ 통과하면 JwtTokenProvider가 userId를 담아 토큰 생성 (24시간 만료).
26. ④ 프론트가 토큰을 받아 localStorage 등에 저장.
27. ⑤ 이후 요청마다 axios 인터셉터가 Authorization: Bearer {token} 헤더를 자동 첨부.

28. ⑥ Spring Security의 JwtAuthenticationFilter가 토큰을 검증해 userId를 꺼낸 뒤 Authentication 객체를 만들어 SecurityContext에 저장.
29. ⑦ Controller에서 Authentication.getPrincipal() 또는 @AuthenticationPrincipal로 현재 사용자를 꺼냄.

7-5. 우리 코드와 맞춰보기

```
// AuthController
@GetMapping("/me")
public ResponseEntity<UserDto> getMe(Authentication authentication) {
    Long userId = (Long) authentication.getPrincipal();
    User user = authService.getUserById(userId);
    return ResponseEntity.ok(UserDto.from(user));
}
```

Authentication은 Spring Security가 필터에서 토큰을 검증한 뒤 만들어 놓은 객체입니다.

컨트롤러는 그걸 받아 "누구인지"를 알 수 있습니다.

Tip. JWT 토큰 만료 시간이 지나면 401을 반환합니다. 프론트 axios 인터셉터에서 401을 감지해 자동으로 로그인 페이지로 보내는 패턴을 자주 씁니다.

8장. React SPA — 페이지 이동이 "가짜"인 이유

8-1. 전통 방식과의 차이

기존 웹은 페이지를 이동할 때마다 서버에서 HTML을 새로 받아 브라우저를 통째로 다시 그렸습니다. SPA는 처음 한 번만 받고, 이후 "이동"은 JavaScript가 일부 화면을 갈아끼웁니다. 장점은 속도와 부드러운 UX.

8-2. Flora의 App.jsx가 하는 일

```
<AuthProvider>
  <Router>
    <Layout>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/products" element={<ProductListPage />} />
        <Route path="/products/:id" element={<ProductDetailPage />} />
        ... (20여 개 라우트)
      </Routes>
    </Layout>
    <ChatBot />
  </Router>
</AuthProvider>
```

- Router: URL을 감시하며, 경로가 바뀌면 그에 맞는 element를 렌더.
- Layout: 헤더·네비게이션·푸터 같은 공통 영역.
- AuthProvider: 로그인 상태를 전역으로 내려주는 React Context.
- ChatBot: 라우트 밖에 두어 어느 페이지에서나 떠 있음.

8-3. 컴포넌트·상태·이펙트

React의 3대 개념만 익히면 대부분의 흐름이 보입니다.

개념	역할	대표 API
컴포넌트	화면 조각. 함수가 JSX를 반환.	function Button() { return <button>...</button>; }
상태 (State)	변하는 값. 바뀌면 재렌더.	const [qty, setQty] = useState(1);
이펙트 (Effect)	렌더 후 부가 작업(데이터 fetch, 구독 등)	useEffect(() => { fetch... }, [deps]);

8-4. 데이터 가져오기 — ProductListPage 흐름

30. 페이지 렌더 직후 useEffect가 실행.

31. useEffect 안에서 productApi.list()를 호출 → axios가 GET /api/products.
32. 응답 JSON을 setProducts(...)로 상태에 반영.
33. 상태가 바뀌면 React가 해당 컴포넌트를 다시 렌더 → 화면에 상품 목록 표시.

8-5. AuthContext — 로그인 상태 공유

Context는 "앱 전체에 값을 내려주는 기능"입니다. 각 컴포넌트가 props를 타고 타고 내려받지 않아도 됩니다.

```
// 사용 예
const { user, login, logout } = useAuth();
if (!user) return <Navigate to="/login" />;
```


9장. axios 인터셉터 — 모든 요청에 토큰이 붙는 마법

axios는 HTTP 클라이언트 라이브러리입니다. fetch보다 편리한 기능이 많은데, 그 중 가장 유용한 것이 인터셉터입니다. 요청이 나가기 직전과 응답이 도착한 직후에 가로채어 공통 처리를 할 수 있습니다.

```
// src/api/index.js 예시
import axios from 'axios';

const api = axios.create({ baseURL: '/api' });

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      // 토큰 만료 시 로그인 페이지로
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default api;
```

이렇게 단 한 곳에서 토큰 첨부과 공통 에러 처리를 모두 해결합니다. 팀 컨벤션 "직접 fetch() 쓰지 말고 이 api 인스턴스만 사용"의 이유입니다.

10장. DB 설계 원리 — Oracle과 MongoDB를 함께 쓰는 감각

10-1. 관계형 DB의 기본기

Primary Key (PK) — 행을 유일하게 식별하는 열. Flora에선 각 테이블의 id 필드.

Foreign Key (FK) — 다른 테이블의 PK를 가리키는 열. 예: ORDERS.user_id → USERS.id.

정규화 — 중복을 줄이고 일관성을 높이기 위해 테이블을 분리하는 과정.

역정규화 — 읽기 성능이나 구현 편의를 위해 일부 중복을 허용하는 것. Flora의 POSTS.likeCount 카운터가 좋은 예.

인덱스 — 자주 조회되는 컬럼에 붙이는 "찾아보기". 느린 WHERE/JOIN을 빠르게 만든다.

10-2. Flora가 23→10으로 줄인 이유

초안은 교과서적으로 정규화된 23개 테이블이었습니다. 하지만 초기 MVP에서는 개발 속도와 유지보수가 더 중요했기 때문에 다음처럼 통합했습니다.

원본 (초안)	통합 결과	의사결정 근거
USERS + OAUTH + BADGES + TERMS	USERS	사용자 한 명의 속성이니 하나로 모음
ORDERS + ORDER_ITEMS + PAYMENTS	ORDERS	MVP는 단일 상품 주문만 지원
COMMUNITY + NOTICES + LIKES	POSTS + likeCount	category 필드로 구분, 좋아요는 카운터
PLANT_CALENDARS + GROWTH_DIARIES	PLANT_CALENDARS	식물 관리와 일기를 한 타임라인으로
SUBSCRIPTIONS + ANNIVERSARY_DELIVERIES	SUBSCRIPTIONS	정기구독·기념일 배송을 단일 엔티티로

△ **주의.** 이건 완벽한 설계가 아니라 "지금 단계에 맞는" 설계입니다. 예컨대 좋아요 기록을 추적하려면 다시 LIKES 테이블을 만들어야 합니다. 이런 trade-off를 명확히 인지하는 것이 중요합니다.

10-3. MongoDB를 쓰는 경우

plants(식물 도감)와 plant_disease_diagnoses(병해 진단)은 Mongo에 있습니다.

- plants: 식물마다 관리 항목이 다릅니다. 선인장은 "물주기 간격"이 중요하지만, 난초는 "습도"가 중요. Map<String, Object>로 careInfo 필드를 두어 스키마를 유연하게.
- plant_disease_diagnoses: AI 응답 형식이 모델을 바꾸면 달라질 수 있음. 고정 스키마로 묶으면 매번 마이그레이션이 필요.

10-4. 두 DB 연결 — plantId 문자열 bridge

Oracle의 PRODUCTS.plantId, PLANT_CALENDARS.plantId, SUBSCRIPTIONS.plantId는 모두 MongoDB plants._id(문자열)을 가리킵니다. DB 간 FK는 걸 수 없으니 애플리케이션 단에서 무결성을 관리합니다.

```
// 상품 상세 조회 시
ProductDto product =
productRepository.findById(id).map(ProductDto::from).orElseThrow();
if (product.getPlantId() != null) {
    Plant plant =
plantRepository.findById(product.getPlantId()).orElse(null);
    product.setPlantInfo(plant);
}
return product;
```

11장. End-to-End 추적 — "상품을 장바구니에 담는다"

1~10장을 모두 한 번에 엮는 실전 추적입니다. 이 한 가지 시나리오를 완전히 이해하면, 다른 기능도 같은 패턴으로 읽을 수 있습니다.

① 사용자 액션

상품 상세 페이지에서 "장바구니" 버튼 클릭. React 컴포넌트의 onClick 핸들러가 실행됩니다.

② 프론트: API 호출

```
// ProductDetailPage.jsx
const handleAddToCart = async () => {
  try {
    await cartApi.addItem({ productId: id, quantity: qty });
    alert('장바구니에 담겼습니다. ');
  } catch (e) {
    alert('로그인이 필요합니다. ');
  }
};
```

cartApi.addItem 내부는 api.post('/cart', payload)를 호출합니다.

③ 네트워크 — 토큰 첨부

axios 인터셉터가 localStorage의 JWT를 꺼내 Authorization 헤더에 붙인 뒤 전송.

④ 백엔드: 보안 필터

Spring Security의 JwtAuthenticationFilter가 토큰을 검증하고 userId를 SecurityContext에 저장.

⑤ 백엔드: Controller

```
@PostMapping
public ResponseEntity<CartItemDto> add(
    @RequestBody CartRequest req,
    Authentication auth) {
    Long userId = (Long) auth.getPrincipal();
    return ResponseEntity.ok(cartService.add(userId, req));
}
```

⑥ 백엔드: Service

```
public CartItemDto add(Long userId, CartRequest req) {
    User user = userRepository.findById(userId).orElseThrow();
    Product p = productRepository.findById(req.getProductId()).orElseThrow();
    if (p.getStockQuantity() < req.getQuantity())
        throw new IllegalStateException("재고 부족");
}
```

```
CartItem item = cartItemRepository
    .findByUserAndProduct(user, p)
    .map(it -> { it.addQuantity(req.getQuantity()); return it; })
    .orElse(CartItem.builder().user(user).product(p).quantity(req.getQuantity())
        .build());
return CartItemDto.from(cartItemRepository.save(item));
}
```

포인트: 동일 상품이면 수량만 더하고, 처음이면 새 CartItem 생성. 재고 체크도 여기서.

⑦ DB: Repository가 SQL 실행

JPA가 INSERT 또는 UPDATE 쿼리를 자동 생성해 Oracle에 보냅니다.

⑧ 응답: 역순으로 올라감

Service → Controller에서 DTO로 감싼 뒤 JSON으로 직렬화 → axios가 받아 프론트로 반환 → React state 업데이트 → 화면 갱신.

Tip. 이 시나리오에서 하나라도 실패하면(재고 부족, 상품 없음, 토큰 만료 등) 각 계층이 예외를 던집니다. Spring의 @ControllerAdvice로 공통 예외 핸들러를 두면 HTTP 상태 코드를 일관되게 내려줄 수 있습니다.

12장. 외부 API 연동 — 산림청 · OpenAI · PG

12-1. 우리가 쓰는 외부 API

서비스	용도	관련 Service 클래스
산림청 공공데이터	식물 기본 정보(학명 · 특징 · 관리법)	ForestApiService.java
OpenAI	챗봇 응답, 꽃 추천	OpenAiService.java
Pixabay	식물 이미지 검색	PixabayService.java
Plant.id (계열)	사진 → 식물 식별 / 병해 진단	(연관 서비스)
카카오/토스페이	결제 (예정)	ORDERS.pgTransactionId 필드로 연결

12-2. 호출 패턴

우리 백엔드의 ____ApiService들은 공통적으로 다음 패턴을 따릅니다.

34. application.properties에 API 키 저장 (절대 Git에 올리지 않도록 .gitignore).
35. RestTemplate 또는 WebClient로 외부 URL에 요청.
36. 응답 JSON을 자바 객체로 매핑.
37. 필요하면 결과를 MongoDB에 캐싱 (특히 이미지나 도감 정보).

△ **주의.** API 키는 절대 프론트 코드에 두지 마세요. 브라우저 개발자 도구로 누구나 볼 수 있습니다. 프론트는 우리 백엔드에 요청하고, 백엔드가 자기 서버에서 외부 API를 호출해 중계해야 합니다.

13장. 자주 나오는 용어 & 공부 로드맵

13-1. 30개 핵심 용어 치트시트

용어	한 줄 설명
HTTP	클라이언트-서버 통신 규약
REST	URL+메서드로 자원을 다루는 API 설계 원칙
JSON	웹에서 데이터를 문자열로 주고받는 형식
SPA	페이지 전환을 JS로 처리하는 단일 페이지 앱
SSR/CSR	렌더링을 서버에서 하나 브라우저에서 하나의 구분
JWT	서명된 토큰에 사용자 정보를 담는 인증 방식
Stateless	서버가 이전 요청의 상태를 기억하지 않는 구조
CORS	다른 도메인 API 호출을 허용/차단하는 규칙
ORM / JPA / Hibernate	객체 ↔ 테이블 자동 매핑 기술
Entity / DTO	DB 매핑 객체 / API 전송용 객체
Repository	DB 접근 전용 계층(인터페이스)
Service	비즈니스 로직 전용 계층
Controller	HTTP 입구 전용 계층
DI	의존성을 직접 new하지 않고 프레임워크가 주입
Bean	Spring이 관리하는 객체(싱글톤)
Transaction	여러 DB 작업을 원자적으로 묶음
N+1 문제	목록 순회 중 연관 엔티티 조회가 N번 추가로 나가는 문제
인덱스	조회 가속을 위한 DB 자료구조
정규화 / 역정규화	중복 제거 설계 / 성능을 위한 중복 허용
PK / FK	기본키 / 외래키
NoSQL	고정 스키마가 없는 저장소(문서·키밸류 등)

용어	한 줄 설명
Singleton	프로세스 전체에서 인스턴스가 하나
Lombok	@Getter/@Builder 등을 자동 생성하는 자바 라이브러리
Axios interceptor	요청/응답 전처리 후킹
React Context	전역 상태를 props 없이 내리는 기능
useEffect	렌더 후 실행되는 사이드이펙트 훅
Swagger / OpenAPI	API 자동 문서화 도구
CI/CD	테스트·배포 자동화 파이프라인
CORS Preflight	OPTIONS 요청으로 본 요청 가능 여부 확인
환경변수	코드 밖에 두는 설정 값(API 키 등)

13-2. 각자 역할별 다음 공부 추천

백엔드 담당

- Spring Security 필터 체인 순서
- JPA 영속성 컨텍스트 / 1차 캐시 / dirty checking
- @Transactional의 전파 옵션과 롤백 조건
- Spring의 AOP와 @ControllerAdvice 예외 핸들링
- 테스트: MockMvc, @DataJpaTest

프론트엔드 담당

- React 렌더링 원리(VDOM, reconciliation)
- useEffect 의존성 배열과 클린업
- useMemo/useCallback, 커스텀 훅
- React Query 또는 SWR — 서버 상태 관리
- 접근성(a11y)과 폼 검증

DB·설계 담당

- EXPLAIN PLAN으로 쿼리 읽기
- 인덱스 설계 원칙과 커버링 인덱스

- 트랜잭션 격리 수준과 동시성 이슈
- MongoDB 인덱스 · aggregation pipeline

13-3. 모르는 걸 만났을 때 전략

38. 에러 메시지를 통째로 복사해 검색 → 공식 문서 먼저 확인.
39. 코드에서 문제가 생긴 지점을 3계층(Controller/Service/Repository) 중 어디인지 먼저 분리.
40. 브라우저 Network 탭으로 요청 · 응답 JSON을 직접 확인 (요청이 제대로 나갔는지, 응답 상태 코드가 무엇인지).
41. 백엔드 콘솔 로그로 SQL 쿼리 실행 여부 확인 (application.properties의 show-sql: true).
42. 되는 케이스와 안 되는 케이스의 차이를 비교해 변수를 좁힌다.

13-4. 마무리

Flora는 3-Tier 구조 위에 JWT 인증, 3계층 백엔드, React SPA, 이중 DB라는 현대 웹 서비스의 전형적인 구성을 모두 담고 있는 작은 축소판입니다. 이 한 프로젝트를 처음부터 끝까지 제대로 이해하면, 이후 다른 서비스 코드를 봐도 같은 패턴이 반복된다는 것을 깨닫게 될 거예요. 한 챕터씩 천천히, 실제 코드를 켜놓고 따라가 보시기 바랍니다.

— Team 積み木, Flora 프로젝트 정리노트